i have to do some data syncing.
DynamicTable,DynamicColumn and DynamicCellData are 3 foreign tables in the DynamicTable schema in the ReadModelProcessor database which users update. These foreign tables physically reside in ManagedDB database. I have azure service bus process which reads those foreign table data and sync user data in the "DynamicTable"."0a04b818-91da-40f6-a022-3a1cbb6e8a42" table in ReadModelProcessor database which named after "GId" from DynamicTable table. All dynamic tables are referenced using TableId which is an Id in DynamicTable table.

Becasue of some technical issue in the service bus, some data in those foreign table or may be a whole table with data are missing in the DynamicTable Schema in the ReadModelProcessor database. I have upload the data structure of one table for understanding. There are approximately 2000 tables. Can you create a script which will check if all the "GId" has been created in the ReadModelProcessor database or not. If not then create them. Also, count compare the row data between Foreign table DyncamicCellData and inside the GId tables. If data missing then sync the data.

Ask me any question that you need to do this.

Show less

Used psql-code-reviewer skill.

I'll inspect the uploaded CSV structures first, then draft a PostgreSQL-safe reconciliation script. I'll assume PostgreSQL/foreign tables unless the file structure shows otherwise, and I'll call out anything that needs confirmation.

From the sample, the target GId table has 3,503 rows, while `DynamicCellData` has 715 distinct row numbers. That means the sample looks like the foreign cell data is incomplete relative to the target, not the other way around. I'll still provide a repair script that inserts rows missing from each GId table based on `DynamicCellData`, but it should not delete target rows.

Thought for 1m 17s

I created a PostgreSQL reconciliation script here:

Download `reconcile_dynamic_tables.sql`

What it does:

1. Checks every `"DynamicTable"."DynamicTable"."GId"` table.
2. Creates the missing GId table in schema `"DynamicTable"` if it does not exist.
3. Adds missing columns from `"DynamicTable"."DynamicColumn"` if the table exists but columns are missing.
4. Compares expected row count from `"DynamicTable"."DynamicCellData"` using distinct `"RowNumber"`.
5. Inserts rows that exist in `DynamicCellData` but are missing from the GId table.
6. Optionally fills existing NULL cells from `DynamicCellData`.
7. Writes results into a temp audit table: `pg_temp.dynamic_table_reconcile_audit`.

Run it first in dry-run mode:

SQL

```sql
BEGIN;

CALL "DynamicTable".reconcile_gid_tables(true, NULL, true);

SELECT *
```

```
ROLLBACK;
```

Then execute for real:

SQL

```sql
BEGIN;

CALL "DynamicTable".reconcile_gid_tables(false, NULL, true);

SELECT *
FROM pg_temp.dynamic_table_reconcile_audit
ORDER BY table_id;

COMMIT;
```

For one table only:

SQL

```sql
CALL "DynamicTable".reconcile_gid_tables(false, 832, true);
```

Important finding from your sample files: the uploaded GId table has **3,503 rows**, but `DynamicCellData` has only **715 distinct RowNumber values**. So for that sample, `DynamicCellData` appears to be missing data compared with the GId table. The script therefore only inserts/fills from `DynamicCellData`; it does **not delete** extra rows from the GId table.

Before using this in production, I need you to confirm these points:

1. Should the script repair only missing rows/cells, or should it also overwrite mismatched values in existing GId tables?
2. Are target dynamic tables always in schema `"DynamicTable"` inside `ReadModelProcessor`?
3. Should only active tables be processed, for example `DynamicTable.StatusId = 1`?
4. Are target columns always named from `DynamicColumn.Name`, not from `DynamicColumn.GId`?
5. How should `"Created By"`, `"Updated By"`, `"Created Date"`, and `"Updated Date"` be populated when there is no matching `DynamicCellData` row for those columns?

My recommendation: run this first on a restored backup or staging copy, because it creates tables and inserts data across approximately 2,000 dynamic tables.


so that, we are on the same page: This is how the schema looks like

Thought for 4s

Yes, we're on the same page now.

Your `ReadModelProcessor` database has:

```
ReadModelProcessor
└── Schemas
    └── DynamicTable
        ├── Foreign Tables
        │   ├── DynamicCellData
        │   ├── DynamicColumn
        │   └── DynamicTable
        │
        └── Tables
            ├── 0a04b818-91da-40f6-a022-3a1cbb6e8a42
            ├── 0a5fcb77-9409-4398-b01a-8a0279dc9f55
            ├── ...
            └── many GId-named physical tables
```

So the key point is:

```
"DynamicTable"."DynamicTable"      -- foreign table metadata
"DynamicTable"."DynamicColumn"     -- foreign table column metadata
"DynamicTable"."DynamicCellData"   -- foreign table cell data

"DynamicTable"."<GId>"             -- real local physical table in ReadModelProcessor
```

That matches the script direction: compare metadata/cell data from the three foreign tables against the local physical GId tables in the same `"DynamicTable"` schema.

One thing I would change/check in the script before you run it: it must distinguish **foreign tables** from **ordinary local tables**. For checking whether a GId table exists, it should look in `information_schema.tables` or `pg_class`, but it should only treat the GId-named objects as target physical tables. The three metadata tables should remain foreign tables and should not be recreated.

The existence check should effectively be like:

SQL

```sql
SELECT EXISTS (
    SELECT 1
    FROM pg_class c
    JOIN pg_namespace n ON n.oid = c.relnamespace
    WHERE n.nspname = 'DynamicTable'
      AND c.relname = target_gid::text
      AND c.relkind IN ('r', 'p') -- ordinary table or partitioned table
);
```

And the three source objects are:

SQL

```sql
"DynamicTable"."DynamicTable"
"DynamicTable"."DynamicColumn"
"DynamicTable"."DynamicCellData"
```

The local target object is dynamic:

SQL

```sql
format('%I.%I', 'DynamicTable', gid::text)
```

Given your screenshot, I would **not** use a separate schema for synced data. Everything is under the same `"DynamicTable"` schema, with foreign metadata tables and local GId tables living side by side.

Before I revise the script, I need one important confirmation:

When creating a missing GId table, should the table structure be built from:

```
DynamicColumn.Name
```

as real column names?

For example, if `DynamicColumn` has:

```
TableId = 832
Name = Customer Name
```

should the generated table contain:

```sql
"Customer Name" text
```

or is there another field, such as `GId`, `Key`, `SystemName`, or `NormalizedName`, that should be used as the physical column name?

also,
-- Foreign Server: prddb_accountreadview

-- DROP SERVER IF EXISTS prddb_accountreadview

CREATE SERVER prddb_accountreadview
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host 'dev001dbgre20.postgres.database.azure.com', dbname 'ManagedDB', port '6432', fetch_size '50000', async_capable 'true');

Show more

Thought for 50s

Perfect — this confirms the physical target table structure.

I updated the reconciliation script to match your DDL more closely:

Download updated script: `reconcile_dynamic_tables_v2.sql`

Key changes based on what you shared:

1. The three source objects stay as **foreign tables**:

```sql
"DynamicTable"."DynamicTable"
"DynamicTable"."DynamicColumn"
"DynamicTable"."DynamicCellData"
```

2. The target GId tables are **local physical tables** under the same schema:

```sql
"DynamicTable"."<GId>"
```

3. When checking whether a GId table exists, the script now only accepts local table types:

```sql
c.relkind IN ('r', 'p')
```

So it will not confuse foreign tables with target local tables.

4. Newly created GId tables follow your provided structure:

```sql
"RowNumber" bigint NOT NULL,
<DynamicColumn.Name> character varying COLLATE pg_catalog."default",
"OrganizationId" bigint,
PRIMARY KEY ("RowNumber")
```

5. Dynamic values are inserted as text/varchar only. I removed the earlier typed casting for `DateTime`, `Enum`, numeric, etc., because your actual target table stores even `"Date"`, `"Price"`, `"Created Date"`, etc. as `character varying`.

Use dry run first:

```sql
BEGIN;

CALL "DynamicTable".reconcile_gid_tables(false, NULL, true, 'lobdev');

SELECT *
FROM pg_temp.dynamic_table_reconcile_audit
ORDER BY table_id;

ROLLBACK;
```

Execute for real:

```sql
BEGIN;

CALL "DynamicTable".reconcile_gid_tables(true, NULL, true, 'lobdev');

SELECT *
FROM pg_temp.dynamic_table_reconcile_audit
ORDER BY table_id;

COMMIT;
```

For one table only:

```sql
BEGIN;

CALL "DynamicTable".reconcile_gid_tables(true, 832, true, 'lobdev');

SELECT *
FROM pg_temp.dynamic_table_reconcile_audit
ORDER BY table_id;

COMMIT;
```

One important note: your source server points to `ManagedDB`, and the foreign tables read from `ManagedDB.DynamicTable.*`. The script does not create or change the FDW server or foreign tables. It only uses them as the source and repairs the local physical GId tables inside `ReadModelProcessor`.

can i download this conversation in a pdf format for reference.

Thought for 50s

Yes. I created a PDF reference copy of this conversation:

Download the conversation PDF